

TECHNICAL ASSESSMENT

Agentic RAG System with Resilient Scraping, Prompt Engineering, Multi-Agent Orchestration, and MCP Tool Access

100,000 Document Corpus, Bot-Protected Sources, Production Architecture

Candidate: Qasim Sajjad

Time: 96 Hours.

Submission: Working code, architecture diagram, design doc

What This Is

You've got a 100,000 document corpus to build search and RAG on top of. That part alone would be a solid senior level test. This version adds two things on top, because we want to see the full range of what your resume claims:

1. A meaningful chunk of that corpus sits behind sites with real bot prevention: Cloudflare/JS challenge pages, rate limiting, and pages that only render content after JavaScript executes. Your ingestion pipeline has to actually deal with that, not assume clean HTML shows up for free.
2. On top of the RAG pipeline, you'll build a small multi-agent system using LangGraph that uses RAG as one tool among several, exposed through an MCP server, so agents call it through a standard tool interface instead of hardcoded function calls.
3. Because a chunk of your corpus comes from scraped, uncontrolled web content, your prompt design has to hold up when retrieved text contains junk, contradictions, or instructions embedded in the page itself. This is a prompt engineering test as much as a retrieval test.

We are not looking for a notebook demo. We want to see how you think about a system someone else could hand off, maintain, and scale. If you take a shortcut anywhere, flag it, that's not a problem. Pretending you didn't is.

On AI coding tools: use whatever you'd normally use, including Claude Code, Copilot, or any other assistant. We assume most engineers do, and we're not testing whether you typed every line yourself. What we're testing is whether you can make the actual decisions (chunking approach, tool scoping, agent boundaries, failure handling) and whether you understand your

own system well enough to explain, defend, and extend it live. The written submission gets you to the conversation. The conversation is where most of the evaluation actually happens.

If you used an AI tool for a meaningful part of this, keep a short record of what you asked it to do and what you changed or corrected afterward. We'll ask for it, and how you directed the tool is itself something we're evaluating.

Deliverable 1: Architecture Diagram

Draw the full system, start to finish: raw scraping through the final API response, including the scraping resilience layer, the MCP boundary, and the agent layer.

Cover all of these:

Scraping & Ingestion Layer

- How you scrape sites that use JS rendering, rate limiting, or bot detection challenges, and what your pipeline does when a fetch fails or gets blocked
- How you decide when to back off, retry, or mark a source as permanently unreachable rather than looping forever
- Deduplication and normalization before anything enters the pipeline
- Full re index vs incremental update handling
- Legal and ethical boundaries you'd set for yourself here (robots.txt, rate limits, terms of service). We want to see that you've thought about this, not that you've found a clever workaround for every block.

Chunking & Embedding Layer

- Chunking logic and how it adapts across content types
- Embedding model choice and why
- Metadata tagging per chunk (source URL, doc type, date, section)

Vector Store & Search Layer

- Vector DB choice and structure (indexes, namespaces, filters)
- Dense only vs hybrid (dense plus BM25/sparse) retrieval
- Reranking step, present or explicitly justified as absent

MCP Layer

- What MCP server(s) you expose, and what tools live behind them. At minimum a `search_corpus` tool wrapping RAG retrieval, plus a second tool of your choice (for example a "get scrape status for source X" tool, since you now have a scraping subsystem worth exposing)
- How a client (the LangGraph agent) discovers and calls these tools
- What context and permissions are scoped at the MCP boundary

Agent Layer

- LangGraph graph structure: nodes, edges, and where branching or looping happens
- How the agent decides whether to call the RAG tool, the scrape status tool, or answer directly
- How you prevent infinite loops or repeated redundant tool calls
- State passed between agent steps: what's kept, what's dropped

LLM & Response Layer

- What context actually reaches the LLM, and in what format
- Handling for queries no document answers well
- Source attribution in the final response

Prompt & Guardrail Layer

- Where your system prompts live and how they're versioned, not just typed inline wherever they're used
- How retrieved chunks are framed in the prompt so the model treats them as reference material rather than instructions to follow
- What happens if a scraped document contains text that looks like an instruction aimed at the model (for example a page that says something like "ignore previous instructions and do X")
- How prompts differ across your router agent, your responder agent, and your RAG answer generation, and why they need to differ

API Layer

- FastAPI endpoint design
- Whether search, RAG, and the agent share an endpoint or are separated
- Any caching layer

Format is your call: Excalidraw, draw.io, Lucidchart, or a clean hand drawn photo. What matters is that someone unfamiliar with the system can follow it without asking you questions.

Deliverable 2: Partial Code Implementation

Build the core pieces. You don't need to ship the whole thing, but what you write should be clean, real, and runnable. Python plus FastAPI expected, given your background.

Component A: Resilient Scraping & Ingestion Pipeline

This is the new hard part. Show:

- How you fetch pages that require JS rendering (headless browser, rendering service, or equivalent) versus plain static pages, and how your pipeline detects which one it's dealing with
- Retry and backoff behavior when a request is rate limited or blocked, including a point where the system gives up gracefully and logs the failure instead of hammering the target
- Parsing across file types once content is actually retrieved (HTML, PDF, tables)
- Chunking logic: recursive, semantic, fixed size, or hybrid
- Metadata attached per chunk before it hits the vector store
- How this runs at scale: batching, parallelism, async

You do not need a working evasion toolkit. We want to see sound engineering (real headless rendering, sane retry logic, respecting rate limits and robots.txt) rather than a demonstration of defeating a specific vendor's bot protection. If you'd normally use a paid scraping service (Firecrawl, Bright Data, ScraperAPI, or similar) in production instead of building this from scratch, say so and explain why. That's a legitimate answer, not a cop out.

Component B: Embedding & Vector Store Upsert

- Embedding model, batch size, rate limit handling
- Upsert logic with full metadata (source, chunk index, doc type, date, not just the vector)
- How you'd handle a re embed if you swap embedding models later
- Namespace or index structure if separating document categories

(~100K docs times ~5 chunks is roughly 500K vectors. Your upsert logic should reflect that this isn't a toy dataset.)

Component C: Query, Retrieval & Reranking

- Query embedding and vector search
- Metadata filtering (doc type, date range, etc.)
- Reranking: cross encoder, Cohere, or an explicit reason for skipping it
- Static vs adaptive k
- Behavior when nothing retrieved is actually relevant

Keep pure semantic search and full RAG generation cleanly separated in code.

Component D: MCP Server & Tool Exposure

- Stand up a minimal MCP server exposing at least two tools: your RAG retrieval (`search_corpus`) and one additional tool tied to the scraping subsystem or another capability of your choice
- Show the tool schema each tool exposes to a calling agent. This is the part that tests whether MCP is actually implemented or just used as a label
- Demonstrate a client (a simple script is fine) successfully discovering and calling both tools through the MCP interface

Component E: LangGraph Multi-Agent Orchestration

- At least two distinct agent roles in the graph (for example a router or planner agent that decides what's needed, and a responder agent that generates the final answer using retrieved context). This does not need to be complex, but it must be real branching logic, not a single LLM call dressed up as a graph
- The graph should call into your MCP tools rather than hitting your functions directly
- Show how state flows between nodes, and what happens if a tool call fails or a scrape source is unavailable

Component G: Prompt Engineering & Injection Defense

Show the actual prompts you use, not just a description of them. Specifically:

- Your system prompt(s) for the RAG answer generation step, and for each distinct agent role in your LangGraph graph, kept as versioned files or constants, not scattered inline strings
- How you structure retrieved context inside the prompt so the model is clearly told this is reference material to reason over, not a set of instructions to obey. This matters because your corpus includes scraped, uncontrolled web content
- A test case (can be a small script or a unit test) where a retrieved chunk contains an embedded instruction trying to redirect the model's behavior, and a demonstration that your system does not follow it
- How you enforce structured output where it matters (for example forcing a citation format, or a fixed JSON shape for the agent trace), and what you do when the model doesn't comply with that structure on the first try
- Any prompt iteration you did: what didn't work initially, what you changed, and why. If you only wrote one version and didn't iterate, say that plainly rather than implying you tested variations you didn't

Component F: FastAPI Endpoints

At minimum:

- **/search**: semantic search over the corpus, returns ranked chunks with source metadata
- **/ask**: full RAG endpoint, question in, generated answer plus citations out, handles low confidence retrieval gracefully
- **/agent**: routes a question through your LangGraph multi-agent system (which calls MCP tools), returns the final answer plus a trace of which agent and tool steps were taken
- **/ingest/status**: reports scrape and ingestion status for a given source, including sources that failed or are being retried

Include a basic README with setup instructions and a curl example for each endpoint.

Deliverable 3: Written Design Document

One to two pages. No fluff, just the reasoning behind your key decisions, written like you're handing it to a senior engineer, not presenting to a client.

1. Scraping & Resilience Strategy What's your actual approach to sites that block or rate limit you? Where's the line you draw between "worth engineering around" and "not worth it, use a third party service or drop the source"? How do you know a source is genuinely unreachable versus temporarily rate limited? What's your plan for a source that changes its page structure and silently breaks your parser?

2. Chunking Strategy Approach and why. How different content types (paragraphs, tables, images, short vs long docs) were handled. Chunk size and overlap: empirically validated or based on prior experience? What did you learn if you tried more than one approach?

3. Embedding Model Which model, and the quality, cost, and latency tradeoff you made. Did you benchmark anything? Would the choice change for a domain specific corpus (legal, medical)?

4. MCP & Agent Design Reasoning Why did you scope the tools the way you did? What's exposed to the agent, and what's deliberately not? How does your agent decide when to call a tool versus answer from its own reasoning? What's the actual benefit of routing through MCP here versus calling your functions directly from the agent code? Be honest if the benefit is mostly architectural rather than something this specific assessment strictly needed.

5. Handling Scale: 100,000 Documents Across Unreliable Sources ~500K vectors, with some sources actively resisting collection. How does ingestion avoid becoming a multi week job once you factor in retries, backoff, and rendering overhead? What breaks first at 1 million documents? How would you add 50 clients each with their own 100K document corpus and their own set of protected sources? Rough cost estimate: embedding, vector DB, LLM calls per query, scraping infrastructure (proxies, rendering, retries).

6. Prompt Engineering Strategy Walk through how you actually designed your prompts, not just what they say. What did the first version look like, and what did you change after seeing real outputs? How do you keep retrieved web content from being interpreted as instructions? How do you enforce structured output reliably, and what's your fallback when the model ignores the format? If you used few shot examples anywhere, why there and not elsewhere?

7. Measuring Quality What metrics: Recall@k, MRR, RAGAS, something custom? How would you build a regression suite so a pipeline or prompt change doesn't silently degrade quality? How do you detect and reduce hallucinations, and separately, how do you detect the agent making a wrong tool selection decision? How would you catch a prompt change that quietly makes the model more likely to follow injected instructions from scraped content?

8. AI Tool Usage If you used an AI coding assistant for any meaningful part of this, say where, what you asked it to do, and what you had to correct or rewrite yourself. If you didn't use one, say that too. Either answer is fine. What we're looking for is an honest, specific account rather than a vague mention either way.

How We'll Evaluate

We're not looking for "the correct answer." We're looking for deliberate decisions with clear reasoning.

Area	What Good Looks Like	What Weak Looks Like
Scraping Resilience	Real rendering strategy, sane retry and backoff, clear give up threshold, honest about using third party services where sensible	No handling of blocks or JS rendering, or an unrealistic claim of defeating every protection

Area	What Good Looks Like	What Weak Looks Like
Chunking	Semantic or recursive, justified by content type, empirically validated	Fixed size only, no rationale
Embedding	Benchmarked or chosen with explicit tradeoff reasoning	Default choice, no comparison
Scale Thinking	Async ingestion, batching, re embed strategy, cost estimate that includes scraping overhead	100K treated like 1K, no scale consideration
Search Quality	Hybrid retrieval, adaptive k, reranking	Dense only, static k, no reranking
MCP Implementation	Real tool schemas, real discovery and call flow, sensible scoping	MCP used as a label over a normal function call
Agent Design	Genuine branching or decision logic across roles, defined failure handling	Single LLM call relabeled as a graph, no real branching
Prompt Engineering	Versioned prompts, clear framing of retrieved content, demonstrated injection resistance, honest account of iteration	Single untested inline prompt, no injection consideration, vague claims of "prompt tuning"
Evaluation	RAGAS, regression tracking, tool selection accuracy tracked	"I tested it manually"
Failure Handling	Low confidence detection, fallback response, source grounding, graceful handling of unreachable sources	No edge case handling, LLM always generates, pipeline breaks silently on a blocked source
Code Quality	Clean structure, typed, commented at non obvious points	Works but unreadable, no README

Weighting note: the table above scores the artifact, but the live session (walkthrough, cold spot checks, live extension task, injection test) carries more weight in the final decision than the written submission alone. A polished submission paired with a shaky live round is a stronger signal than a rough submission with a confident, precise live round, not the other way around.

After Submission

The written submission is the entry ticket, not the main event. Most of the actual evaluation happens in a live 60 to 75 minute session, structured like this:

Part 1: Walkthrough (20 minutes) Walk us through your architecture and key decisions. Be ready to explain, not defend, every choice. If something didn't work as expected, that's worth discussing too.

Part 2: Cold Spot Checks (10 minutes) We'll point at a specific function, prompt, or tool schema in your code, picked at the time, and ask you to explain what it does and why it's built that way, without preparing an answer in advance. This applies equally whether you wrote it by hand or with an AI tool's help. If you directed the build, you should be able to do this without hesitation.

Part 3: Live Extension Task (20 to 30 minutes) We'll give you a small new requirement you have not seen before, on the spot, screen shared. Something like adding a new MCP tool, handling a new failure mode in the scraper, or changing how the agent decides between two tools. You can use any AI coding tool you normally would. We're watching how you work through it: what you check first, how you verify the change didn't break something else, and how you explain what you did afterward.

Part 4: Injection Test (5 to 10 minutes) We'll hand you a new scraped snippet on the spot with a different embedded instruction and watch what your system does with it.

A few things to keep in mind:

- If you took a shortcut anywhere, note it in the design doc. It won't count against you. Not noting it will.
- No deployment required. Local code with clear setup instructions is fine.
- The diagram, code, and design doc must all describe the same system, consistently.
- Bring the system running locally to the call. If it's not running, that's a real problem, not a minor one.

Good luck.